

Git 高级协作 workflow 完全指南

Git 高级协作 workflow 完全指南

文档类型: 工具进阶指南 适用读者: 中级 - 高级开发者 前置知识: Git
基础操作 (commit/push/pull/branch) 最后更新: 2026-07-02**文档类型**: 工具进阶指南
适用读者: 中级 - 高级开发者
前置知识: Git 基础操作 (commit/push/pull/branch)
最后更新: 2026-07-02

文档类型: 工具进阶指南 适用读者: 中级 - 高级开发者 前置知识: Git
基础操作 (commit/push/pull/branch) 最后更新: 2026-07-02**文档类型**: 工具进阶指南
适用读者: 中级 - 高级开发者
前置知识: Git 基础操作 (commit/push/pull/branch)
最后更新: 2026-07-02

1. 分支策略

1.1 Git Flow vs GitHub Flow vs Trunk-Based

维度	Git Flow	GitHub Flow	Trunk-Based
核心分支	main + develop	仅 main	仅 main (trunk)
Feature 分支	从 develop 创建	从 main 创建	短期 (< 1 天)
Release 分支	release/`release/`	用 Tag	Release Flag
Hotfix 分支	hotfix/`hotfix/`	直接修	Cherry-pick
发布频率	按版本周期	持续部署	持续部署
适用团队	版本化软件	SaaS 服务	高节奏团队

1.2 分支命名规范

```
#
feature/SHOP-1234-add-cart-checkout      # Jira  +
bugfix/SHOP-2345-fix-price-rounding      # Bug
hotfix/SHOP-3456-critical-auth-patch     #
release/v2.3.0                           #
chore/update-eslint-config                #
docs/api-v2-migration-guide              #

#
my-branch
fix2
test
wip
```

2. Commit 规范2. Commit 规范

2.1 Conventional Commits2.1 Conventional Commits

```
<type>(<scope>): <description>

[optional body]

[optional footer]
```

支持的 Type:

TypeType	说明说明	示例示例
feat`feat`	新功能新功能	feat(order): add order cancellation
fix`fix`	Bug 修复Bug 修复	fix(payment): handle zero-amount ed
docs`docs`	文档变更文档变更	docs(api): update rate-limit sectio
style`style`	代码格式代码格式	style: format with prettier 3.x`sty
refactor`refactor`	重构重构	refactor(auth): extract token valid
perf`perf`	性能优化性能优化	perf(query): add composite index fo
test`test`	测试测试	test(order): add parallel concurren
chore`chore`	构建/工具构建/工具	chore(deps): bump axios to 1.6.0`ch
ci`ci`	CI/CD	ci: add staging deploy workflow`ci:

2.2 Commit Message 反例2.2 Commit Message 反例

```
# Commit Message
git commit -m "fix"
git commit -m "update"
git commit -m "wip"
git commit -m "fix bug "
git commit -m "."
git commit -m "asdfasdf"

# Commit Message
git commit -m "fix(payment): prevent duplicate charge on network retry

When the payment provider responds with a network timeout, the client
may retry the request. This fix adds an idempotency key check to
prevent creating duplicate charges for the same order.

Closes SHOP-2345"
```

3. 高级 Git 操作3. 高级 Git 操作

3.1 Interactive Rebase 实战3.1 Interactive Rebase 实战

```
# 5 commit
git rebase -i HEAD~5

# rebase
# pick = commit
# reword = message
# squash = commit message
# fixup = commit message
# drop = commit
# edit =
```

```
pick a1b2c3d feat(order): add order creation API
squash b2c3d4e fix(order): address review comments #
pick c3d4e5f feat(order): add order list API
reword d4e5f6g feat(order): add order detail API # message
squash e5f6g7h fix(order): fix pagination bug #
pick f6g7h8i docs(order): add API documentation
```

3.2 Cherry-Pick 选择合并3.2 Cherry-Pick 选择合并

```
# commit
git checkout release/v2.2
git cherry-pick a1b2c3d

# commit
git cherry-pick a1b2c3d e5f6g7h i9j0k1l

# start end
git cherry-pick a1b2c3d..e5f6g7h

#
git cherry-pick --continue
# cherry-pick
git cherry-pick --abort
```

3.3 Git Bisect 二分定位 Bug3.3 Git Bisect 二分定位 Bug

```
#
git bisect start
git bisect bad HEAD          #
git bisect good v2.0.0       #

# Git commit
#
git bisect bad  #
#
git bisect good  #

# Git Bug commit
# a1b2c3d is the first bad commit

#
git bisect reset
```

4. 合并策略4. 合并策略

4.1 Merge vs Rebase vs Squash4.1 Merge vs Rebase vs Squash

策略策略	历史记录历史记录	适用场景适用场景	注意事项注意事项
git merge`git merge`	保留完整分支历史保留完整分支	合并长期分支（release → mai	产生 merge commit产生 merge commit
git rebase`git rebase`	线性历史线性历史	Feature 分支同步 main 更新Fe	不要 rebase 已推送的 commit不要 rebase 已
git merge --squash`git merge`	单 commit 合并单 commit	合并 Feature 合并到 main（简洁历	丢失分 commit 信息丢失分 commit 信息

4.2 冲突解决策略4.2 冲突解决策略

```
# ours
git merge feature/new-ui -s ours

# +
git merge feature/new-ui -X ours      #
git merge feature/new-ui -X theirs   #

#
git mergetool --tool=vscode

#
git merge --abort
```

5. 团队协作最佳实践5. 团队协作最佳实践

5.1 Pull Request 规范5.1 Pull Request 规范

```
## PR
[SHOP-1234] feat: add shopping cart checkout flow

## PR

###
→ →

### Issue
- Closes SHOP-1234
- Related to SHOP-1200

###
- [ ] (`npm test`)
- [ ] (`npm run test:e2e`)
- [ ] //
- [ ] ESLint / TypeScript

### /
UI

###
- `idempotency_key`
```

5.2 Code Review 关注点5.2 Code Review 关注点

关注维度关注维度	检查要点检查要点
正确性正确性	逻辑是否正确？边界条件是否覆盖？逻辑是否正确？边界条件是否覆盖？
安全性安全性	是否存在注入/越权风险？是否存在注入/越权风险？
性能性能	是否有 N+1 查询？是否有不必要的循环？是否有 N+1 查询？是否有
可维护性可维护性	代码是否清晰易懂？命名是否合理？代码是否清晰易懂？命名是否合理？
测试覆盖测试覆盖	核心逻辑是否有测试？边界用例是否覆盖？核心逻辑是否有测试？边界用例是否

6. Git Hooks6. Git Hooks

6.1 常用 Hook 场景6.1 常用 Hook 场景

```
# .git/hooks/pre-commit
#!/bin/sh
# lint-staged

npx lint-staged

# .git/hooks/commit-msg
#!/bin/sh
# commit message

COMMIT_MSG_FILE=$1
COMMIT_MSG=$(cat "$COMMIT_MSG_FILE")

PATTERN="^(feat|fix|docs|style|refactor|perf|test|chore|ci)(\\(\\.+\\))?: .+"

if ! echo "$COMMIT_MSG" | grep -qE "$PATTERN"; then
    echo " Commit message  Conventional Commits "
    echo ": type(scope): description"
    exit 1
fi

# .git/hooks/pre-push
#!/bin/sh
#

echo "Running tests before push..."
npm test -- --bail
```

6.2 Husky + lint-staged 配置6.2 Husky + lint-staged 配置

```
{
  "husky": {
    "hooks": {
      "pre-commit": "lint-staged",
      "commit-msg": "commitlint -E HUSKY_GIT_PARAMS",
      "pre-push": "npm test -- --bail"
    }
  },
  "lint-staged": {
    "*.{ts,tsx}": ["eslint --fix", "prettier --write"],
    "*.{json,md,yaml}": ["prettier --write"]
  }
}
```

7. 常见问题与恢复7. 常见问题与恢复

7.1 常见误操作恢复7.1 常见误操作恢复

误操作误操作	恢复命令恢复命令	说明说明
错误的 commit错误的 commit	git reset --soft HEAD~1`git reset -	撤销 commit，保留更改撤销 commit，保留更改
错误的 amend错误的 amend	git reflog + git reset`git reflog`	找回之前的 commit找回之前的 commit
删除未合并分支删除未合并分支	git reflog + git checkout -b`git re	30 天内可恢复30 天内可恢复
强制推送覆盖他人强制推送覆盖他人	git push -f 前确认! `git push -f 前确认!	无法直接恢复无法直接恢复
误删文件误删文件	git checkout HEAD -- <file>`git che	恢复到最后一次提交状态恢复到最后一次提交状态
Merge 到错误分支Merge 到错误分支	git reset --merge ORIG_HEAD`git res	撤销最近一次 merge撤销最近一次 merge

7.2 reflog 救命指南7.2 reflog 救命指南

```
# reflog HEAD
git reflog

#
# abc1234 HEAD@{0}: commit: feat(order): add checkout
# def5678 HEAD@{1}: commit: fix(cart): handle empty state
# ghi9012 HEAD@{2}: reset: moving to HEAD~3

# 3
git reset --hard HEAD@{2}

#
git checkout -b recovered-branch HEAD@{5}
```

8. 总结8. 总结

掌握 Git 高级工作流是团队高效协作的关键。核心原则：掌握 Git

高级工作流是团队高效协作的关键。核心原则：

1. 清晰的提交历史是团队的共同资产**清晰的提交历史**是团队的共同资产
2. Conventional Commits 让变更意图一目了然**Conventional Commits** 让变更意图一目了然
3. Rebase 用于保持 Feature 分支同步，Merge 用于合并长期分支**Rebase** 用于保持 Feature 分支同步，**Merge** 用于合并长期分支
4. reflog 是最后的救命稻草，善用它**reflog** 是最后的救命稻草，善用它

提高建议：每周花 30 分钟练习一次交互式 rebase，直到操作完全熟练。 **提高建议**：每周花 30 分钟练习一次交互式 rebase，直到操作完全熟练。

提高建议：每周花 30 分钟练习一次交互式 rebase，直到操作完全熟练。 **提高建议**：每周花 30 分钟练习一次交互式 rebase，直到操作完全熟练。

本文档采用 CC BY-SA 4.0 许可。欢迎提交 PR 补充更多技巧。